

Discussion Module:

DSE Search, Geospatial

Learn to deliver DSE Geospatial queries. E.g.,
Where's the nearest Starbucks-

Prerequisites to this Discussion Module:

- That you are comfortable with making and using DSE Search chained analyzers.
- That you are comfortable with DSE Search filter queries, facets, and the extended parser syntax of dismax, or edismax.
- Why (above) ? Not because we use these objects per se. Comfort with these topics means DSE Search geospatial objects and query syntax will be familiar.

Discussion Lab:

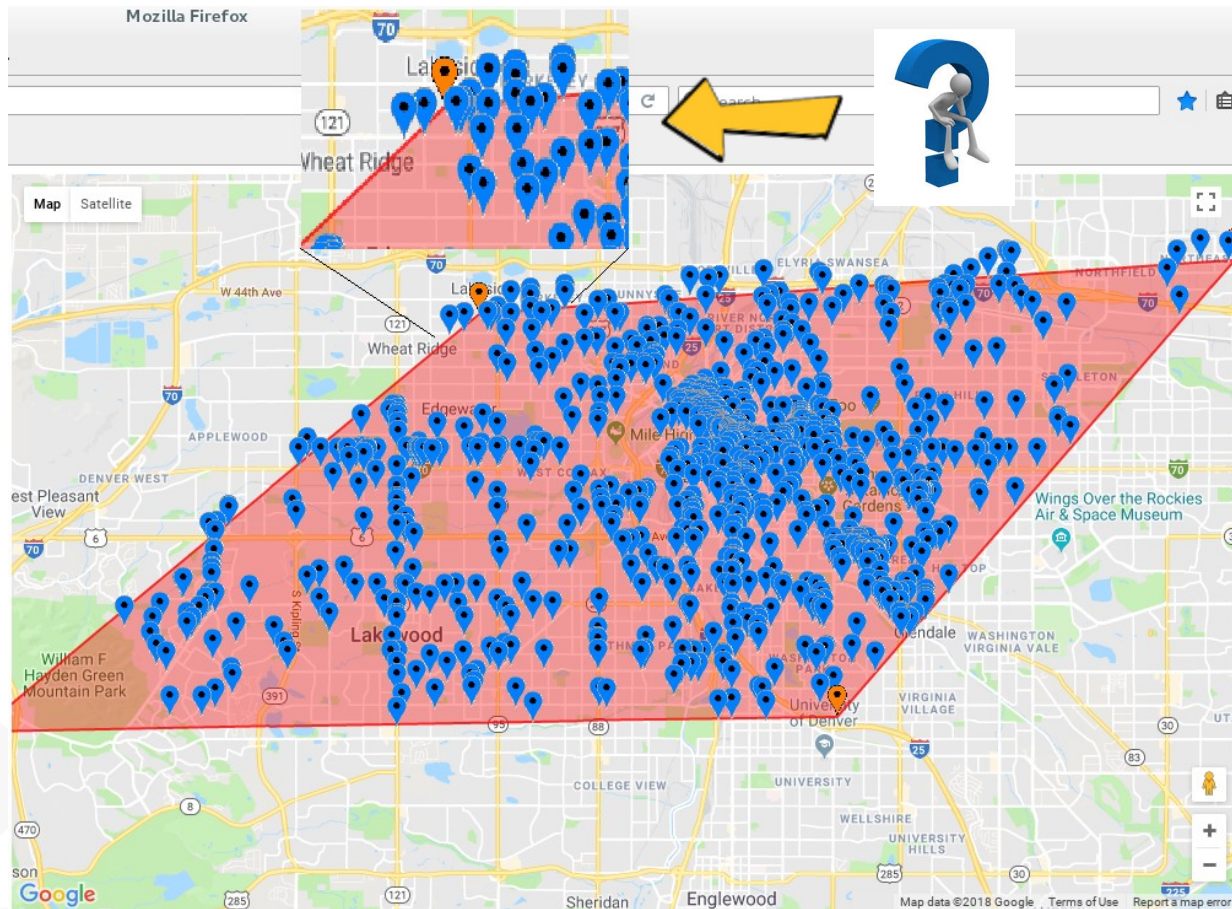
Open Probe

Next page: Why are there points returned from a DSE Search geospatial query that display outside of the search area ?

Geospatial versus Spatial

Geospatial; spherical mathematical model, (the Earth is nearly a sphere). Distance calculated relative to curved surface.

Spatial; Cartesian, two dimensional mathematical model



End of Discussion Lab:

What Makes a Query Geospatial, Spatial, (other) ?

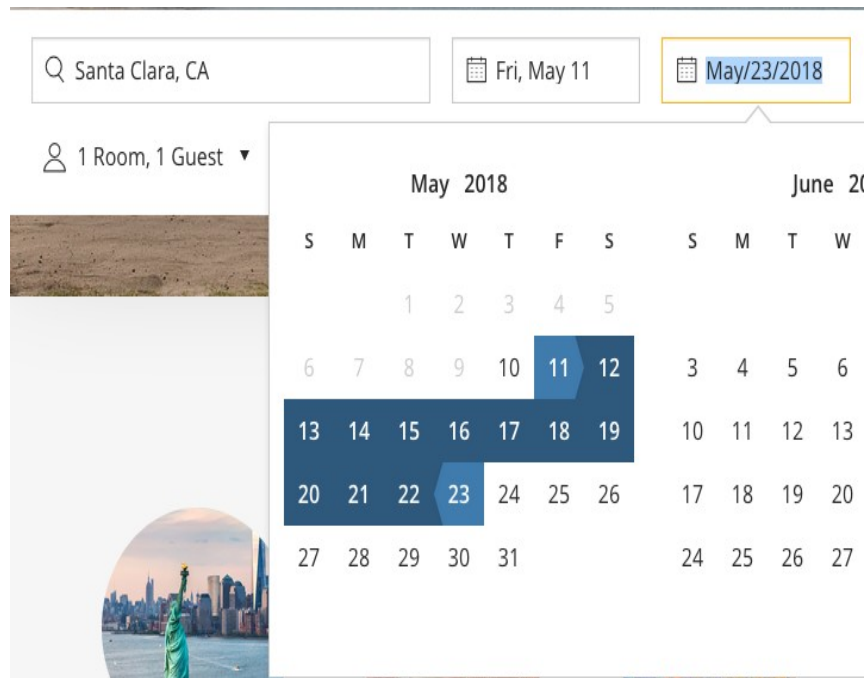


Source: <https://spherelandma508.wordpress.com/about/>

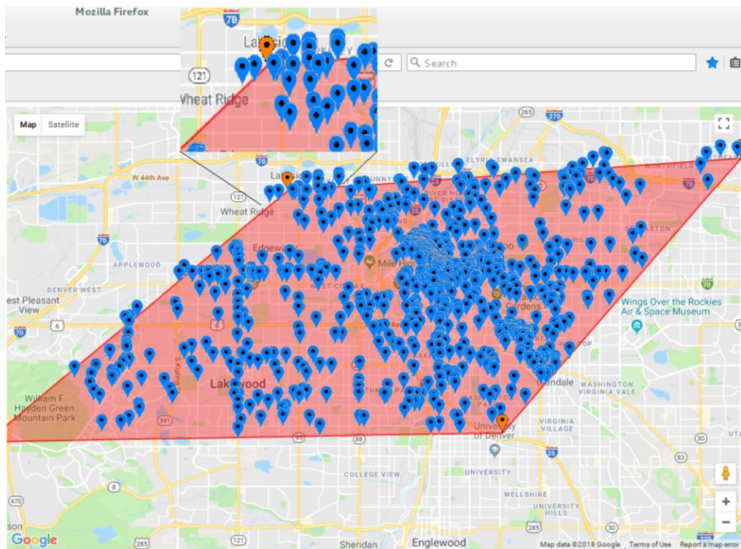
- Field types
 - No analyzer needed (no text to parse).
 - Common/past use of `dynamicField`. E.g., Take a Lat/Lon pair, and index each decimal value.
 - Basically, extended common numeric types.
- To query: New predicate expressions
- But the answer is: Field Types

Geospatial versus Spatial

- Geospatial- The Earth is not perfectly round (huge mountains, deep valleys); don't care, but still better than imagining the world is flat. Account for international date line.
- Spatial; points sit atop a flat plane, x/y coordinate pair
 - How far are products on either side of a warehouse
 - Also, time series; start and end dates/time



Choices: (Spatial Strategy)



- RDBMS: volatile data, B-Tree+
- DSE: Web scale data sizes, hash (and)

Geospatial

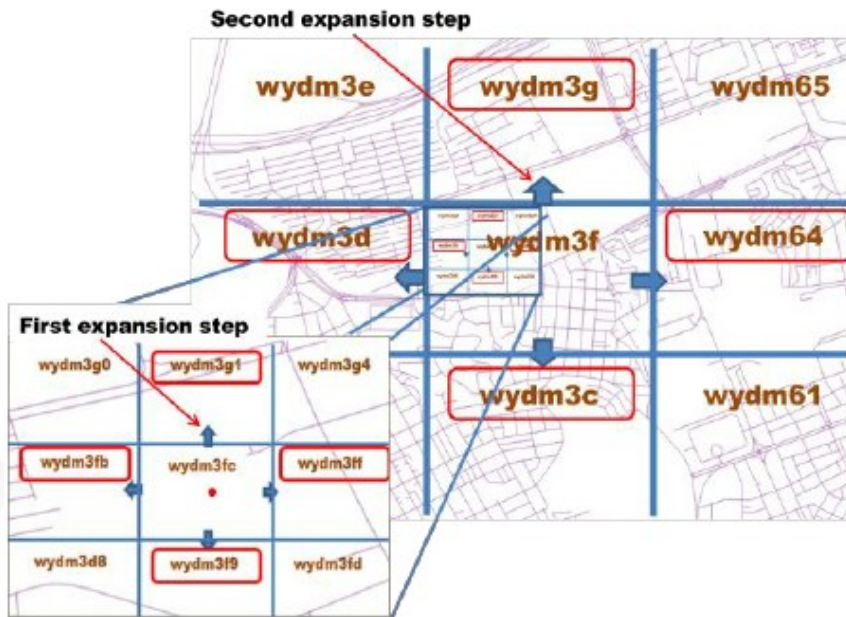
- Distance from point ?
- Intersection of boundaries of geometric shapes ?
- Other
- distErrPact, distErr ? (configurable, tunables)
- 2D polygon on 3D world ?, 3D polygon on 3D world ?

Coarse Description of Geohash-

- Take the Earth, and divide it into 32 equal sized rectangles.
- Take each of the rectangles above, and divide those into 32 (further, sub) rectangles.
- After you iterate the above cycle 4 times, you have divided the Earth into distinct regions as accurate as 12 meters.
- By storing a unique string only 5 characters long, and an array of whatever items are located in this distinct (144 meter square region of the Earth), you can serve a lot of geo-spatial queries quickly and easily.
- Further, if you also index not only the 5 character string, but all substrings of the 5 character string, you have many highly performant choices.



Coarse Description of Geohash, Example-



If a 5 character string that tells us that Boston/MA/USA equates to the string constant, DRY2Y, we should also index the sub-strings of DRY2Y, those being; D, DR, DRY, DRY2 (DRY2Y already being indexed).

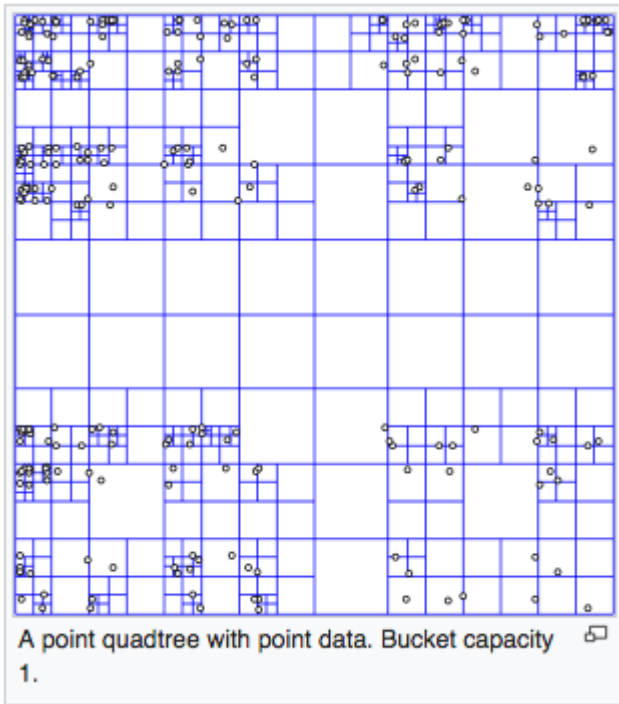
When we need to find all points within 100 miles/km of the point, DRY2Y, we need only look for the equality, DRY (or whatever the actual accuracy is).

Geohash and/or Quadtree

DSE Search geospatial/spatial uses geohash, or also quadtree.

For now we state,

- DSE Search uses geohash for geodetic data only (geo-spatial only).
- DSE Search uses quadtree for either geo-spatial or spatial data, including temporal data (time series data).
- Implemented as TermsEnum; highly performant, allows set operands (unions, intersections, other)



Solr Filter (Left Edge) N-gram filter ?

Index Analyzer

org.apache.solr.analysis.KeywordTokenizerFactory {luceneMa

position	1
term text	Enterprise
startOffset	0
endOffset	10

org.apache.solr.analysis.LowerCaseFilterFactory {luceneMa

position	1
term text	enterprise
startOffset	0
endOffset	10

org.apache.solr.analysis.EdgeNGramFilterFactory {maxGran

position	1	2	3	4	5	6	7	8
term text	ent	ente	enter	enterp	enterpr	enterpri	enterpris	enterprise
startOffset	0	0	0	0	0	0	0	0
endOffset	3	4	5	6	7	8	9	10

org.apache.solr.analysis.EdgeNGramFilterFactory {maxGran

position	1	2	3	4	5	6	7	8	9	10	11	12	13	14
term text	ent	nte	ente	ter	nter	enter	erp	terp	nterp	enterp	rpr	erpr	terpr	nter
startOffset	0	1	0	2	1	0	3	2	1	0	4	3	2	1
endOffset	3	4	4	5	5	5	6	6	6	6	7	7	7	7

Now you know a use case for left edge N-Gram; break a token like DRY2Y into further tokens, D, DR, DRY, ..

By preindexing (indexing) each of the sub-elements of the token (DRY2Y), you allow for fast filtering of ranges.

Must reads on Geospatial-

David Smiley at MITRE on Apache Solr spatial/geospatial is available at,

- <https://www.youtube.com/watch?v=L2cUGv0Rebs>
- <https://www.slideshare.net/lucenerevolution/lucene-solr-4-spatial-extended-deep-dive>

The Shape interface

(may become an abstract class in the next version)

- interface Shape {
 - Point **getCenter**();
 - Rectangle **getBoundingBox**();
 - boolean **hasArea**();
 - double **getArea**();
 - SpatialRelation **relate**(Shape other);
 - Must support Point & Rectangle
- enum *SpatialRelation*
 - DISJOINT, INTERSECTS, WITHIN, CONTAINS
 - Note: simpler set than the "DE-9IM" spatial standard
 - no "equals" or "touches"



- Index a Lat/Lon pair
- Search for rows within a point-radius (circle), or bounding box

- Sort results by distance from said point.

End to End Example:

Distance from point, radius, bounding box, arbitrary rectangle.

- DSE cluster, with Search enabled
- Authority to make tables, other
- CQLSH/Studio access-
- Example uses field type LatLonType, which is most common



Distance from Point: CREATE KEYSPACE

```
DROP KEYSPACE IF EXISTS ks_7453;  
CREATE KEYSPACE ks_7453 WITH  
REPLICATION =  
    {'class': 'SimpleStrategy',  
     'replication_factor': 1};  
USE ks_7453;
```

```
CREATE TABLE t1  
(  
    col1          TEXT,  
    my_latlong    TEXT,  
    PRIMARY KEY ((col1))  
);
```



Distance from Point: Add Data, CREATE INDEX

```
INSERT INTO t1 (col1, my_latlong)
VALUES ('1', '38.8623758053, 106.214532852');
```

```
VALUES ('2', '38.8618662055,
-106.212043762');
VALUES ('3', '30.8702532527,
-100.264572144');
VALUES ('4', '-30.8702532527,
-100.264572144');
```

```
CREATE SEARCH INDEX ON t1
WITH COLUMNS col1 ;
```

**OLD
OLD
OLD**



Data type ?

**(Eventually; data integrity
constraint ?**

Distance from Point: Add Field Types



```
ALTER SEARCH INDEX SCHEMA ON t1  
  ADD types.fieldType[@name='ft_latlong',  
    @class='solr.LatLonType',  
    @subFieldSuffix='_coord'];
```

```
ALTER SEARCH INDEX SCHEMA ON t1  
  ADD types.fieldType[@name=  
    'ft_double', @class='solr.TrieDoubleField'];
```

Distance from Point: ADD dynamicField, field

```
ALTER SEARCH INDEX SCHEMA ON t1  
ADD dynamicField[@name=  
  '*_coord', @type='ft_double',  
  @indexed='true', @stored='false'];
```

```
ALTER SEARCH INDEX SCHEMA ON t1  
ADD fields.field[@name=  
  'my_latlong', @type='ft_latlong',  
  @indexed='true', @multiValued='false',  
  @stored='true'];
```

```
RELOAD SEARCH INDEX ON t1;  
REBUILD SEARCH INDEX ON t1;
```



Using Solr Admin UI
Schema Browser:
my_latlong_0_coord
my_latlong_1_coord

Distance from Point: Radius (from circle)



```
SELECT * FROM t1 WHERE solr_query =  
  '{ "q" : "*:*", "fq" : "{!geofilt pt=38.86,-106.21  
    sfield=my_latlong d=10000}" }';
```

col1	my_latlong	solr_query
4	-30.8702532527, -100.264572144	null
3	30.8702532527, -100.264572144	null
2	38.8618662055, -106.212043762	null
1	38.8623758053, -106.214532852	null

Or, '{ "q" : "{!geofilt pt=38.86, ..

What is geofilt ?

- { "q" : "{!edismax qf=col6}Dog ??

{ "q" : "{!geofilt pt=38.86, -106.21 sfield=my_latlong d=10000}" }

It's a query parser. So, expect parameters-

- https://lucene.apache.org/solr/guide/6_6/spatial-search.html
 - d, the radial distance, default km, can set via distanceUnits
 - pt, the center point, "x, y" for Lat/Lon or PointType, "x y" for PRT
 - sfield, the indexed field name
 - (others)
- Other parsers; bbox, (arbitrary rectangle), more.

Distance from Point: Radius (from circle)

```
SELECT * FROM t1 WHERE solr_query =  
{ "q" : "*:*", "fq" : "{!geofilt pt=38.86,  
-106.21 sfield=my_latlong d=1000}" }';
```

```
{ "q" : "*:*", "fq" : "{!geofilt pt=38.86,  
-106.21 sfield=my_latlong d=1}" }';
```

// Same for both

col1	my_latlong	solr_query
2	38.8618662055, -106.212043762	null
1	38.8623758053, -106.214532852	null

Distance from Point: Bounding Box (within square)



```
SELECT * FROM t1 WHERE solr_query =  
{ "q" : "*:*", "fq" : "{!bbox pt=38.86,  
-106.21 sfield=my_latlong d=10000}" }';
```

col1	my_latlong	solr_query
4	-30.8702532527, -100.264572144	null
3	30.8702532527, -100.264572144	null
2	38.8618662055, -106.212043762	null
1	38.8623758053, -106.214532852	null

Distance from Point: (Arbitrary Rectangle)



```
SELECT * FROM t1 WHERE solr_query =  
{ "q" : "*:*", "fq" :  
  "my_latlong:[38,-107 TO 39,-106]" };
```



Negative values are sequenced right
to left (remember the date line)

col1	my_latlong	solr_query
2	38.8618662055, -106.212043762	null
1	38.8623758053, -106.214532852	null

(2 rows)

Distance from Point: Sorting



```
SELECT * FROM t1 WHERE solr_query =  
'{"q":"*:*", "fq": "{!geofilt sfield=my_latlong  
pt=38.86,-106.21 d=30}",  
"sort":"geodist(my_latlong, 38.86,-106.21) asc"}';
```



col1	my_latlong	solr_query
2	38.8618662055, -106.212043762	null
1	38.8623758053, -106.214532852	null

```
"sort":"sum(col3, col4) asc" }'; ??
```

End to End Example:

Distance from point, radius, bounding box, arbitrary rectangle.

REVIEW

- DSE cluster, with Search enabled
 - Authority to make tables, other
 - CQLSH/Studio access-
-
- Example uses field type LatLonType, which is most common



Previously:

- Query from point (in various means)
- Field types,
 - **LatLonType** (what we used; deprecated, use RPT for new work)
 - **PointType**, non-geo version of LatLonType (also deprecated)
 - **LatLonPointSpatialField** (97% backwards compatible with above, but ??)

Adds



- **SpatialRecursivePrefixTreeFieldType** (RPT) & **RptWithGeometrySpatialField**
- **BBoxField**



**Where have we
heard this term
before ?**

https://lucene.apache.org/solr/guide/6_6/spatial-search.html



Sort Changes with Field Type

- The syntax to sort changes when using different field types
- 90+% of examples in world are LatLonType
- Skip LatLonPointSpatialField ??
- Move to RPT-
 - Can support for polygons, but polygons not required
 - No dynamic field required
 - No commas to data
 - Different sort syntax


```
ALTER SEARCH INDEX SCHEMA
```

```
ON t2 ADD types.fieldType[
```

```
@name='ft_latlong',
```

```
@class='solr.SpatialRecursivePrefixTreeFieldType',
```

```
@distErrPct='0.025', @maxDistErr='0.000009',
```

```
@units='degrees'];
```

LatLonPointSpatialField,
other .. sort will change

...

```
SELECT * FROM t2 WHERE solr_query =
```

```
'{"q":":*:*", "fq": "{!geofilt sfield=my_latlong pt=38.86,-106.21 d=30}",
```

```
"sort": "geodist(my_latlong,38.86,-106.21) asc"}';
```

InvalidRequest: Error from server: code=2200 [Invalid query]
message="sort param could not be parsed as a query, and is not a
field that exists in the index: geodist(my_latlong,38.86,-106.21)"

End to End Example:

RPT with Sort

RPT Example

- Can support for polygons, but polygons not required
- No dynamic field required
- No commas to data
- Different sort syntax



Using RPT: CREATE TABLE

**OLD
OLD
OLD**

```
CREATE TABLE t3
(
  col1      TEXT,
  col2      TEXT,
  col3      TEXT,
  col4      TEXT,
  col5      TEXT,
  col6      TEXT,
  col7      TEXT,
  col8      INT,
  PRIMARY KEY ((col1)) );
```

```
CREATE SEARCH INDEX ON t3 WITH COLUMNS col1,
* { excluded : true };
```

Using RPT: INSERT data

```
INSERT INTO t3 (col1, col2, col3, col4,  
col5, col6, col7, col8)  
VALUES ('aaa', '10, 10', '10, 10',  
'10 10', '10 10', '10 10', 'aaa', 1);
```

```
VALUES ('bbb', '11, 11', '11, 11',  
'11 11', '11 11', '11 11', 'bbb', 2);  
VALUES ('ccc', '12, 12', '12, 12',  
'12 12', '12 12', '12 12', 'ccc', 3);  
VALUES ('ddd', '13, 13', '13, 13',  
'13 13', '13 13', '13 13', 'ddd', 4);
```



No comma ?

**OLD
OLD
OLD**

Using RPT: ALTER SEARCH INDEX



```
ALTER SEARCH INDEX SCHEMA ON t3  
  ADD types.fieldType[@name='ft_geo',  
    @class='solr.SpatialRecursivePrefixTreeFieldType',  
    @geo='true', @maxDistErr='0.0'];
```



```
ALTER SEARCH INDEX SCHEMA ON t3  
  ADD fields.field[@name='col6', @type='ft_geo',  
    @indexed='true', @stored='true'];
```

```
RELOAD SEARCH INDEX ON t3;  
REBUILD SEARCH INDEX ON t3;
```

```
SELECT col1, col6 FROM t3
WHERE solr_query='{ "q": "{!func
pt=14,14 sfield=col6 dist=1000}geodist()",
"sort": "score asc"}';
```

```
"sort": "score desc"}';
```

col1	col6
ddd	13 13
ccc	12 12
bbb	11 11
aaa	10 10
col1	col6
aaa	10 10
bbb	11 11
ccc	12 12
ddd	13 13

Using RPT: SELECT



```
SELECT col1, col6 FROM t3 WHERE  
solr_query = '{ "q" : "*:*", "fq" :  
  "{!geofilt score=distance fl=*,  
  score pt=14,14 sfield=col6 d=1000  
  sort=score asc}" }' ;
```

```
sort=score desc}" }' ;
```

```
// Same output for both
```

col1	col6
aaa	10 10
bbb	11 11
ccc	12 12
ddd	13 13

Does not sort correctly-



Moving to Polygons

Use case for polygons

- States/provinces, neighborhoods
- Hexagon most common (they line up neatly)

Polygons requires JTS (Java Topology Suite)

- Small, just shapes
- [Only a thing] because of license

JTS, Java Topology Suite

- Polygons require the installation of the Apache JTS (Java Topology Suite). JTS is detailed on Wikipedia at, https://en.wikipedia.org/wiki/JTS_Topology_Suite

Download the Java Jar (which is how JTS arrives) from,
<http://central.maven.org/maven2/com/vividsolutions/jts/1.13/DSE>



- Search 6.0 supports JTS version 1.13.
- To install JTS, copy the Java Jar file to, /opt/dse/node1/resources/solr/lib when DSE is installed in, /opt/dse/node1
- Super simple; JTS is really just shapes.
- A rolling restart would allow DSE to see and make use of JTS.

Be advised:

- Polygon Query without JTS installed (CQLSH) -

OperationTimedOut: errors={'127.0.0.1': 'Client request timeout. See Session.execute[_async](timeout)'}, last_host=127.0.0.1

- Polygon Query with JTS installed (system.log) -

INFO [IOThread-0] 2018-03-13 14:42:46,427
SolrConfig.java:266 - Found custom filter cache class:
org.apache.solr.search.SolrFilterCache. Use of custom
implementations is an advanced feature, as could cause
bugs and memory leaks.

When using Shapes:

- New query predicates
 - Intersects
 - IsWithin
 - IsDisjointTo
 - Contains

```
SELECT * FROM t3 WHERE solr_query =  
'{ "q" : "*:*", "fq" : "{!field  
f=my_latlong}Intersects(POLYGON((-107 39,  
-107 30, -100 30, -100 39, -107 39)))" }';
```



`{!field ??`

Why does the set of polygon points get wrapped in 2 sets of parenthesis ?

End to End Example:

Polygon query

- DSE cluster, with Search enabled
 - Authority to make tables, other
 - CQLSH/Studio access-
-
- Works with RPT and LatLonType; this example uses LatLonType, which is most commonly seen/documentated



```
CREATE TABLE t3
(
  col1          TEXT,
  my_latlong    TEXT,
  PRIMARY KEY ((col1))
);
```

CREATE TABLE,
data, index



```
INSERT INTO t3 (col1, my_latlong)
VALUES ('1', '38.8623758053, -106.214532852');

VALUES ('2', '38.8618662055, -106.212043762');
VALUES ('3', '30.8702532527, -100.264572144');
VALUES ('4', '-30.8702532527, -100.264572144');
```

```
CREATE SEARCH INDEX ON t3
  WITH COLUMNS col1 {docValues : true };
```

CREATE first of types, dynamicField



```
ALTER SEARCH INDEX SCHEMA ON t3  
  ADD types.fieldType[@name=  
    'ft_double',@class='solr.TrieDoubleField'];
```

```
ALTER SEARCH INDEX SCHEMA ON t3  
  ADD dynamicField[@name=  
    '*_coord', @type='ft_double',  
    @indexed='true', @stored='false'];
```

New field type, polygon capable



```
ALTER SEARCH INDEX SCHEMA ON t3
ADD types.fieldType[@name='ft_latlong',
@class='solr.SpatialRecursivePrefixTreeFieldType',
@spatialContextFactory=
'org.locationtech.spatial4j.context.jts.JtsSpatialContextFactory',
@autoIndex='true', @validationRule='repairBuffer0',
@distErrPct='0.025', @maxDistErr='0.001',
@distanceUnits='kilometers'];
```



```
ALTER SEARCH INDEX SCHEMA ON t3
ADD fields.field[@name='my_latlong',
@type='ft_latlong', @indexed='true',
@multiValued='false', @stored='true',
@docValues='true'];
```

**geo, default true,
means geospatial,
means geohash**

See notes page-

RELOAD SEARCH INDEX ON t3;
REBUILD SEARCH INDEX ON t3;

Query: Intersects POLYGON

```
SELECT * FROM t3 WHERE solr_query =  
'{ "q" : "*:*", "fq" : "{!field f=my_latlong}  
Intersects(POLYGON((-107 39, -107 30,  
-100 30, -100 39, -107 39)))" }';
```

col1	my_latlong	solr_query
3	30.8702532527, -100.264572144	null
2	38.8618662055, -106.212043762	null
1	38.8623758053, -106.214532852	null

(3 rows)

End to End Example:

Polygon query.

REVIEW

- DSE cluster, with Search enabled
- Authority to make tables, other
- CQLSH/Studio access-





End of Module: